

The background of the slide is a dark purple gradient. On the left side, there is a large, intricate, swirling purple ribbon-like shape. Scattered around this shape and in the background are various mathematical symbols and icons in a lighter purple color. These include: $J \equiv$, $D_i \equiv$, $D_j \equiv$, $A2_2$, $> I_3$, $D \equiv$, $P = I_3$, $A_3 =$, $F = \gamma_3$, and a small padlock icon at the bottom left. There are also some small circles and lines connecting some of the symbols.

Number Theory and Cryptography

Number theory is a branch of mathematics. It deals with the properties and relationships of numbers. This presentation explores number theory and its application to cryptography. It covers core concepts such as divisibility and modular arithmetic. We will also examine integer representations and algorithms. Prime numbers, greatest common divisors, solving congruences, and applications of congruences will also be discussed. Finally we will go into cryptography: both classical and public key and the RSA cryptosystem.

 **by Judd Marc**

Divisibility and Modular Arithmetic

Understanding these concepts is crucial in number theory and its applications to cryptography. Divisibility helps in understanding how numbers can be divided evenly, while modular arithmetic plays a key role in encryption and decryption processes. By delving into these topics, one can gain a deeper insight into the foundations of cryptography and its real-world implementation.

1 Divisibility

An integer a is divisible by an integer b (written as $b|a$) if there exists an integer k such that $a = bk$. In other words, b divides a with no remainder.

Example: 12 is divisible by 3 because $12 = 3 * 4$.

2 Division Algorithm

For any integers a and b with $b > 0$, there exist unique integers q (quotient) and r (remainder) such that $a = bq + r$, where $0 \leq r < b$.

Example: If $a = 25$ and $b = 7$, then $25 = 7 * 3 + 4$, where $q = 3$ and $r = 4$.

3 Modular Arithmetic

It involves working with remainders, and it is particularly useful in cryptography for creating secure communication systems. With modular arithmetic, calculations are done within a specific range, providing a way to perform operations that involve cyclical patterns.

Example: $17 \equiv 2 \pmod{5}$ because 17 divided by 5 leaves a remainder of 2.

Divisibility and Modular Arithmetic

Congruences

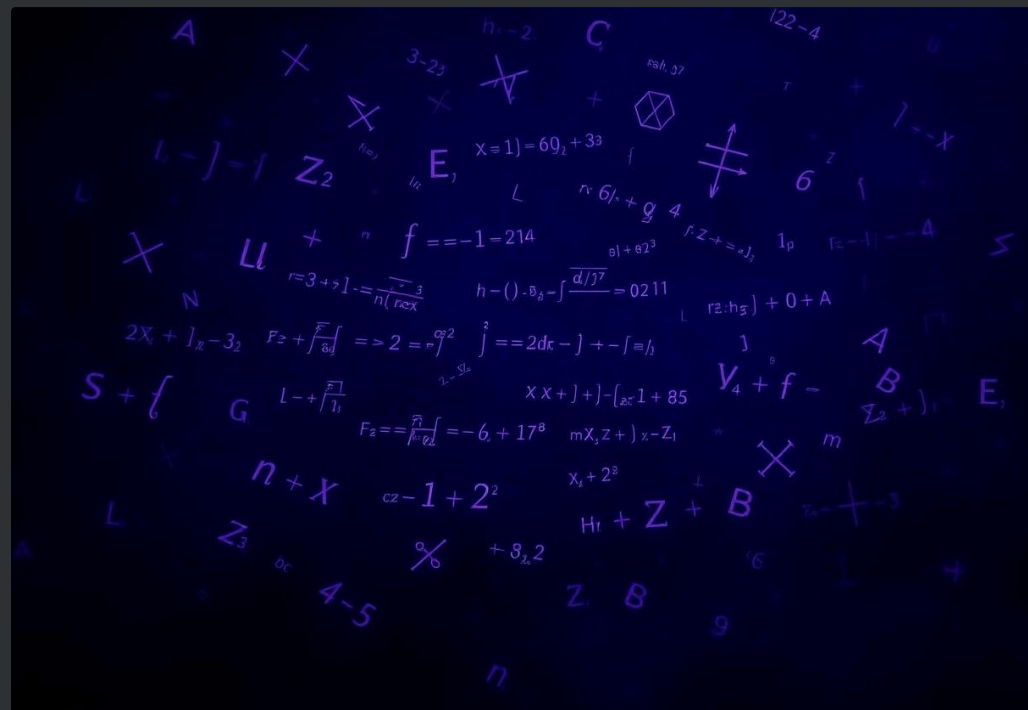
A congruence is a statement that two integers are congruent modulo m . For example, $23 \equiv 3 \pmod{5}$ because both 23 and 3 have a remainder of 3 when divided by 5.

Modular Addition

If $a \equiv b \pmod{m}$ and $c \equiv d \pmod{m}$, then $(a + c) \equiv (b + d) \pmod{m}$. Example: $7 \equiv 2 \pmod{5}$ and $8 \equiv 3 \pmod{5}$, so $(7 + 8) \equiv (2 + 3) \pmod{5}$, which simplifies to $15 \equiv 5 \pmod{5}$, or $0 \equiv 0 \pmod{5}$.

Modular Multiplication

If $a \equiv b \pmod{m}$ and $c \equiv d \pmod{m}$, then $(a * c) \equiv (b * d) \pmod{m}$. Example: $7 \equiv 2 \pmod{5}$ and $8 \equiv 3 \pmod{5}$, so $(7 * 8) \equiv (2 * 3) \pmod{5}$, which simplifies to $56 \equiv 6 \pmod{5}$, or $1 \equiv 1 \pmod{5}$.



Modular arithmetic forms the basis for many cryptographic algorithms. Understanding divisibility rules is crucial for simplifying calculations.

Integer Representations and Algorithms

1 Binary, Octal, and Hexadecimal Representation

Integers can be represented in base 2 (binary), base 8 (octal), and base 16 (hexadecimal). Binary is crucial for digital systems, while octal and hexadecimal offer more compact representations.

Example: The decimal number 10 can be represented as 1010 in binary, 12 in octal, and A in hexadecimal.

2 Decimal Notation

The norm is decimal notation (base 10), which uses ten digits (0-9) to represent numbers.

Example: The number 123 is in decimal notation, where 1 is in the hundreds place, 2 in the tens place, and 3 in the ones place.

3 General Base Representation (Base b)

Any integer can be uniquely represented in base b, where b is a positive integer greater than 1. This is a fundamental concept in number theory and computer science.

Example: The number 29 in base 3 is 1002, since $29 = 1 \cdot (3^3) + 0 \cdot (3^2) + 0 \cdot (3^1) + 2 \cdot (3^0)$.

Conversion Algorithms

Base Conversion Algorithms

- **Decimal to Binary:** Repeated division by 2, record remainders.
- **Binary to Decimal:** Multiply each digit by 2^n , where n is the digit's position.
- **Decimal to Hexadecimal:** Repeated division by 16, record remainders (0-9, A-F).
- **Hexadecimal to Decimal:** Multiply each digit by 16^n , where n is the digit's position.
- **Decimal to Octal:** Repeated division by 8, record remainders.
- **Octal to Decimal:** Multiply each digit by 8^n , where n is the digit's position.



Binary to Octal Conversion

Group binary digits into sets of three from right to left, then convert each group to its octal equivalent.

Example: Binary 101101 becomes Octal 55 (101 101).



Binary to Hexadecimal Conversion

Group binary digits into sets of four from right to left, then convert each group to its hexadecimal equivalent.

Example: Binary 101101 becomes Hexadecimal 2D (0010 1101).



Octal to Binary Conversion

Convert each octal digit to its 3-bit binary equivalent.

Example: Octal 55 becomes Binary 101101.



Hexadecimal to Binary Conversion

Convert each hexadecimal digit to its 4-bit binary equivalent.

Example: Hexadecimal 2D becomes Binary 00101101.

Primes and Greatest Common Divisors

A prime number is only divisible by 1 and itself. Prime numbers are fundamental in cryptography for key generation and encryption algorithms. The greatest common divisor (GCD) of two integers is the largest integer that divides both. Understanding prime numbers and their role in cryptography is essential for securing data and communications. Alongside prime numbers, the concept of the greatest common divisor (GCD) plays a crucial role in ensuring the integrity of cryptographic algorithms. Juxtaposing prime numbers and the GCD highlights their complementary significance in cryptographic practices.

Trial Division Algorithm:

Trial division is a basic method of primality testing. It involves testing whether an integer is divisible by each prime number less than or equal to the square root of the number.

Steps:

1. Take an integer n .
2. Check divisibility by each prime number less than or equal to the square root of n .
3. If n is divisible by any prime number, then n is composite.
4. If n is not divisible by any prime number less than or equal to the square root of n , then n is prime.

*For example, to test if 17 is prime, we only need to check divisibility by primes up to 4 (since $4*4 < 17 < 5*5$), which are 2 and 3. Since 17 is not divisible by 2 or 3, it is prime.*

Prime Factorization:

Every integer greater than 1 can be represented uniquely as a product of prime numbers. This is known as its prime factorization.

Steps:

1. Start with the smallest prime number, 2.
2. Divide the number by the smallest prime number.
3. If the number is divisible by the prime number, then divide again by the same prime number.
4. If the number is not divisible by the prime number, then divide by the next prime number.
5. Repeat until the number is 1.

For instance, the prime factorization of 28 is $2 \times 2 \times 7$, indicating that 28 can be expressed as the product of the prime numbers 2 and 7.

Euclidean Algorithm

The Euclidean algorithm is an efficient method for computing the greatest common divisor (GCD) of two integers. It involves repeatedly dividing the larger number by the smaller number and replacing the larger number with the remainder until the remainder is 0. The GCD is the last non-zero remainder.

Example 1:

Find the GCD of 270 and 192:

1. Divide 270 by 192: $270 = 192 * 1 + 78$ (GCD not yet found)
2. Divide 192 by 78: $192 = 78 * 2 + 36$ (GCD not yet found)
3. Divide 78 by 36: $78 = 36 * 2 + 6$ (GCD not yet found)
4. Divide 36 by 6: $36 = 6 * 6 + 0$ (GCD found)

The GCD of 270 and 192 is 6.

Euclidean

— = G-rate

÷ = Comm

— = 61ivier

0 Division

Linear Combinations

The greatest common divisor (GCD) of a and b can always be expressed as a linear combination of a and b . This is a fundamental concept in number theory and has important applications in solving Diophantine equations and understanding modular arithmetic.

Example:

Let's find the GCD of 270 and 192 and express it as a linear combination.

Euclidean Algorithm

1. Divide 270 by 192: $270 = 192 * 1 + 78$
2. Divide 192 by 78: $192 = 78 * 2 + 36$
3. Divide 78 by 36: $78 = 36 * 2 + 6$
4. Divide 36 by 6: $36 = 6 * 6 + 0$
5. $\text{GCD}(270, 192) = 6$

Linear Combination

1. From step 3: $6 = 78 - 36 * 2$
2. From step 2: $36 = 192 - 78 * 2 \Rightarrow 6 = 78 - (192 - 78 * 2) * 2 \Rightarrow 6 = 78 - 192 * 2 + 78 * 4 \Rightarrow 6 = 78 * 5 - 192 * 2$
3. From step 1: $78 = 270 - 192 * 1 \Rightarrow 6 = (270 - 192 * 1) * 5 - 192 * 2 \Rightarrow 6 = 270 * 5 - 192 * 5 - 192 * 2 \Rightarrow 6 = 270 * 5 - 192 * 7$
4. Therefore: $6 = 270 * 5 + 192 * (-7)$

The GCD of 270 and 192 is 6, and it can be expressed as the linear combination: $6 = 270 * 5 + 192 * (-7)$

$$\frac{1.24.23^2}{3xc} + \frac{2.2.55}{2xc} = +59$$

$$\frac{2.2.131}{10xc} = \frac{1.2.55}{2xc} = +9$$

$$\frac{2.2.2.}{3sc} + nc) = \frac{9.5.5}{2xc} = + \frac{6.6}{2xc} = \frac{2435}{2xc}$$

$$= + \frac{216.5}{2xc}$$

Solving Congruences



Linear Congruences

Solve equations of the form $ax \equiv b \pmod{m}$. The goal is to find an integer x that satisfies the congruence, given integers a , b , and m . This involves finding the modular inverse of a modulo m , if it exists.



Chinese Remainder Theorem

Finds a solution to a system of congruences. Given a system of congruences with pairwise coprime moduli, the Chinese Remainder Theorem guarantees a unique solution modulo the product of the moduli. This is useful in various applications, including cryptography and computer science.



Modular Inverses

Deals with large integers and modular operations. The modular inverse of an integer a modulo m is an integer x such that $ax \equiv 1 \pmod{m}$. Modular inverses are essential for solving linear congruences and performing division in modular arithmetic, especially in cryptography.



Theft upon sale, and
things on for pitant and

Applications of Congruences

image of application of congruences



Hashing Functions

Uses modular arithmetic to map data to a fixed-size range, which is crucial for efficient data retrieval in computer science. Hashing functions convert keys into indices for storing and locating data in data structures like hash tables.

Problem: Risk of collisions, where different keys map to the same index, requiring collision resolution techniques to maintain efficiency.



Pseudorandom Numbers

Generates sequences that appear random but are deterministic, using congruential generators. These generators produce a sequence of numbers based on an initial seed value and modular arithmetic operations.

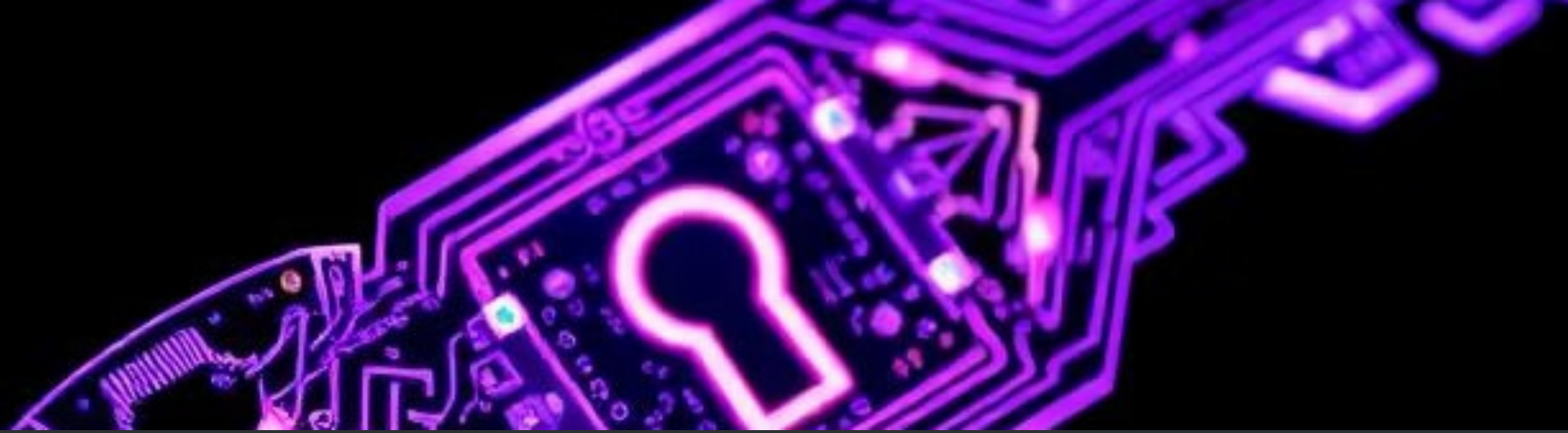
Problem: Sequences are predictable and may exhibit patterns, which can be a security issue in cryptographic applications.



Check Digits

Detects errors in identification numbers (e.g., ISBNs, credit card numbers) using modular arithmetic. A check digit is appended to the number and calculated based on a specific formula that incorporates modular operations.

Problem: Cannot detect all types of errors, especially those that cancel out or involve multiple errors simultaneously.



Cryptography: Classical and Public Key

Classical Cryptography

Classical cryptography is based on substitution and transposition techniques. These methods were used historically. Modern cryptography has moved on.

- Caesar Cipher: Shifts letters by a fixed number.
- Vigenère Cipher: Uses a keyword to encrypt messages.

Symmetric-Key Cryptography

Symmetric-key cryptography uses the same key for encryption and decryption. It relies on secure key exchange.

- AES: Advanced Encryption Standard, widely used.
- DES: Data Encryption Standard, now outdated.

Public-Key Cryptography

Public-key cryptography uses separate keys for encryption and decryption. It relies on mathematical problems.

- RSA: Rivest-Shamir-Adleman, used for secure data transmission.
- ECC: Elliptic Curve Cryptography, used in mobile applications.



RSA Cryptosystem Details

The RSA (Rivest-Shamir-Adleman) cryptosystem is a widely used public-key encryption algorithm. It involves key generation, encryption, and decryption processes. Below are the detailed steps and an example.

1

Key Generation

First, generate the public and private keys:

Choose two distinct prime numbers $p = 61$ and $q = 53$. Compute $n = p * q = 3233$. Calculate $\phi(n) = (p-1) * (q-1) = 3120$. Select an integer e such that $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$; let $e = 17$. Determine d (the modular multiplicative inverse of e modulo $\phi(n)$) such that $d * e \equiv 1 \pmod{\phi(n)}$; $d = 2753$.

2

Encryption

Next, encrypt the message using the public key:

Message $M = 123$ is encrypted using the public key $(n = 3233, e = 17)$ as $C \equiv M^e \pmod{n}$. Therefore, $C \equiv 123^{17} \pmod{3233}$. The ciphertext C is **855**.

3

Decryption

Finally, decrypt the ciphertext using the private key:

Ciphertext $C = 855$ is decrypted using the private key $(n = 3233, d = 2753)$ as $M \equiv C^d \pmod{n}$. Thus, $M \equiv 855^{2753} \pmod{3233}$. The decrypted message M is **123**, recovering the original message.



Cryptographic Protocols

Cryptographic protocols are sets of rules that dictate how cryptographic algorithms should be used. They are essential for secure communication and data protection.

Key Exchange

Key exchange is a fundamental aspect of many cryptographic protocols. It allows parties to securely agree on a shared secret key, which can then be used for symmetric-key encryption.

Example: Diffie-Hellman Key Exchange

Suppose that Alice and Bob want to share a common key:

1. Alice and Bob agree on a prime number $p = 23$ and a base $g = 5$.
2. Alice chooses a secret integer $a = 6$ and calculates $A = ga \bmod p$. So, $A = 56 \bmod 23 = 8$.
3. Bob chooses a secret integer $b = 15$ and calculates $B = gb \bmod p$. So, $B = 515 \bmod 23 = 19$.
4. Alice sends $A = 8$ to Bob, and Bob sends $B = 19$ to Alice.
5. Alice computes the secret key $s = Ba \bmod p$. So, $s = 196 \bmod 23 = 2$.
6. Bob computes the secret key $s = Ab \bmod p$. So, $s = 815 \bmod 23 = 2$.
7. Now, both Alice and Bob have the same secret key $s = 2$, which they can use for secure communication.